

# README

## Single-Hidden-Neuron Neural Network — From Scratch

A single-hidden-neuron, single-output neural network, implemented from scratch in NumPy

(no PyTorch/TensorFlow), predicting Parkinson's disease severity from voice features.

### Dataset

- **Source:** [UCI Parkinson's Telemonitoring dataset](#)
- **Size:** 5,875 voice recordings from 42 patients with early-stage Parkinson's disease
- **Features:** 16 biomedical voice measures (jitter, shimmer, noise-to-harmonics ratio, RPDE, DFA, PPE, etc.) plus patient age
- **Target:** `total_UPDRS` — a continuous clinical score (Unified Parkinson's Disease Rating Scale) quantifying motor symptom severity

### Approach

- **Task:** regression (continuous target), so the output layer uses linear activation (no squashing function) and Mean Squared Error as the loss
- **Columns dropped:** `sex`, `test_time`, and `motor_UPDRS` — the last to avoid data leakage, since it is closely correlated with the target `total_UPDRS`
- **Train/test split:** group-aware split ( `GroupShuffleSplit` , grouped by `subject#` ), so no single patient's repeated measurements appear in both sets
- **Scaling:** features standardized (mean 0, std 1); scaler fit on training data only, then applied to the test set, to avoid leaking test-set statistics into training

- **Model:** forward pass, loss, and backpropagation implemented manually with NumPy  
matrix operations, no autograd

## Architecture

| Layer  | Shape       | Activation    |
|--------|-------------|---------------|
| Input  | 18 features | —             |
| Hidden | 1 neuron    | Sigmoid       |
| Output | 1 neuron    | Linear (none) |

| Weight matrix                       | Shape     |
|-------------------------------------|-----------|
| $W_1$ (input $\rightarrow$ hidden)  | $[18, 1]$ |
| $W_2$ (hidden $\rightarrow$ output) | $[1, 1]$  |

## Results

| Metric                                     | Value              |
|--------------------------------------------|--------------------|
| Test RMSE                                  | 12.44 UPDRS points |
| Baseline RMSE (always predicting the mean) | 8 UPDRS points     |
| $R^2$ score                                | -2                 |

**Diagnosis:** the negative  $R^2$  is explained by sigmoid saturation in the hidden neuron. With 17 summed input features, the pre-activation value  $z_1$  landed far enough from 0 that sigmoid's output clustered almost entirely at its extremes (min  $\approx 8e-08$ , max  $\approx 0.9998$ ) for nearly every sample. Since  $\text{sigmoid\_derivative}(x) = x \times (1 - x)$  approaches 0 at both extremes, this effectively froze the gradient flowing back to initial weight values from the very first few iterations, so the weights never meaningfully moved from their random initialization .

## What I'd try next

- Increase hidden layer size (e.g. 4–8 neurons) to relieve the single-neuron bottleneck

- Switch the hidden layer activation to ReLU to avoid sigmoid saturation
- Tune the learning rate and number of training iterations
- Compare against a PyTorch `nn.Module` implementation of the same architecture

## Concepts & Glossary

The theory behind every step above — classes/objects, tensors, weight initialization, forward pass, gradient descent, backpropagation, and the vanishing gradient problem — is explained for beginners in [CONCEPTS.md](#), with quick term lookups in [GLOSSARY.md](#).